

μ KV: Energy- and Thermal-Aware KV-Cache Eviction for LLM Inference on a Phone

Md Romyull Islam
Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

On a phone the KV cache, not compute, limits sustained LLM decode. But the policies that bound it were designed for server GPUs, and three properties of a real on-device inference engine break them. (i) The cell array is shared across heads and layers, so a cell is freed only if *every* per-head selector dropped it; SnapKV’s nominal per-head budget retains 67.7% of cells on Llama-3.2-1B and 84.9% on Phi-3-mini, and H2O’s 20%-of-context ratio retains 99.8%. Their budgets never materialize on the device. (ii) Reading post-softmax attention forces the FlashAttention-off path, which on device costs more than eviction saves: those policies decode at 0.10–0.19 $\times$ the full cache across three backends. (iii) Compacting through the engine’s state API needs a second full context, exceeding unified memory — Phi-3-mini at 16K is killed by the OS on the phone and fails identically on a 24 GB desktop GPU. We present μ KV, which respects all three: it computes selection scores *inside* the FlashAttention prefill graph through a small side node, so it obtains the attention signal without leaving the fast path; it selects one sequence-level keep-set, so eviction actually frees cells; and it compacts *in place*, sliding survivors into a dense prefix inside the tensors prefill already allocated, so peak memory never rises. On a OnePlus 15, μ KV retains 7.4% of cells and decodes 1.20 $\times$ faster than the full cache while every per-head baseline is 0.10–0.19 $\times$. A faithfully configured StreamingLLM — its own 2004-cell budget, flash-attention on, and the contiguous keep-set its own implementation maintains — also reaches 1.20 $\times$, so keeping FlashAttention on, not evicting more, is what governs on-device speed; μ KV’s advantage over it is that it reaches the same throughput on 2.8 $\times$ fewer cells and retrieves 14/14 needles against its 7/14. Under sustained load μ KV finishes eight back-to-back 4096-token generations in 1.21 $\times$ less wall time at 1.60 $\times$ decode throughput, and in-place compaction makes Phi-3-mini at 16K feasible where the state round-trip is killed. At 64K, μ KV retains 1.4% of the cache at *lower* perplexity than the full cache (10.42 vs 11.05) on a WikiText slice verified disjoint from the prompt. We report costs as plainly as wins: prefill is 1.06 $\times$ slower (the scoring cost); peak resident set is *higher* than vanilla’s, because the KV buffer is allocated at full context up front, so the win is in retained cells and bandwidth, not footprint; a disjoint-slice perplexity shows eviction does not damage language modeling but cannot show that prompt information survives, which is what retrieval measures; and coupling cache size to temperature does not cap heat, so frequency remains the thermal lever and the cache an efficiency lever.

KEYWORDS

Mobile LLM inference, KV cache eviction, FlashAttention, thermal management, energy efficiency, Snapdragon, on-device AI

1 INTRODUCTION

Motivation. Phone LLM workloads differ from a server’s: a single user, a 30–60 s sustained generation, a thermal envelope that throttles long before any compute limit, and an OS that kills the process past a memory-headroom threshold. Quality is necessary but not sufficient; a policy that wins perplexity at the cost of skin temperature, peak resident-set, or battery drain fails deployment. The literature optimizes *one* axis at a time (perplexity, DVFS, or compression alone), but the phone couples all of them: cache size sets DRAM read, read sets power, power sets temperature, temperature sets throttle, throttle sets throughput. A mobile KV-cache policy must be jointly evaluated on *quality* (NIAH, LongBench, perplexity), *throughput* (decode tps at realistic 16K context), and *thermal* (peak DDR vs the kernel cliff, peak skin, watchdog activity), on real hardware. The literature has nothing that does all three on a phone.

Problem. Small instruction-tuned LLMs (1–4B parameters, 4-bit quantized) now run interactively on flagship phones at 5–7 tokens/s on CPU. The remaining wall is not the matrix multiply; it is the KV cache. Once sustained generation passes a thousand tokens, four pressures bind at once:

- **Cache size.** A Phi-3-mini cache at 2K holds about 1.2 GiB (f16 K+V). On a 12 GB phone shared with foreground apps this triggers swap: 158 MB of swap-out under the full cache, swap-in stalls near 280 ms.
- **DRAM bandwidth.** Decode reads the whole cache once per layer per token, roughly 750 MB per attention pass for Phi-3 at 2K. On LPDDR5X (~50 GB/s) this dominates the step.
- **Thermals.** The Qualcomm BCL [17] and the kernel cooling driver cut the big-core clock from 1632 to 883 MHz once DDR passes about 65 °C, often within 90 s of sustained decode.
- **Energy.** A full-cache 2048-token decode draws about 38.7 mAh; on a 5000 mAh battery that is about 130 generations before any other app load.

Five design challenges. A phone-viable evictor must resolve five tensions no single prior mechanism handles together. (i) Any per-head signal from post-softmax attention is locked out of FlashAttention, the kernel that makes mobile decode fast. (ii) q8_0 K quantization only pays above roughly a 1 GB cache, since below that the dequantize cost exceeds the bandwidth saving (§4.4). (iii) Aggressive eviction runs *hotter* at peak, not cooler, because the freed bandwidth converts to clock and power; the envelope is set by peak temperature, not total energy. (iv) A fixed K wastes headroom on short prompts while a fraction-of-prompt K blows the DRAM budget on

long ones. (v) At a tight budget the same ramp tunes either for chat throughput (small K , recent-heavy) or document-QA quality (large K , anchor-heavy), so the split should be chosen per request from a cheap prefill signal. μ KV addresses these with in-graph scoring on the flash-attention path, a precision ablation, steady-state cache reduction, an honest fail-mode disclosure, and an adaptive-anchor dispatcher that sets $\alpha_a \in [0.05, 0.95]$ per prompt from the distant-sharpness fraction of prefill attention (§3).

These pressures are coupled: the system informally minimizes $\alpha_1 \text{Wall} + \alpha_2 \text{Energy} + \alpha_3 \text{Jitter}$ subject to a perplexity budget, a peak-temperature ceiling, and a total-I/O ceiling. One intervention, bounding the cache by a per-head confidence-aware budget, pushes on all four pressures at once.

The policies that could supply that bound were designed elsewhere: H2O [1], TOVA [3], SnapKV [4], StreamingLLM [2], and the per-head allocator Ada-KV [6], all built for server GPUs and measured by perplexity alone. On our hardware canonical H2O is 32% slower than the full cache at decode, because reading attention scores each step forces the FlashAttention-off path.

We test two questions on a real phone. First, whether attention-driven eviction plus a thermal control loop can win on throughput, energy, peak RSS, and thermal margin: yes, and with the mass-based α_a dispatcher on retrieval too, where the earlier count-based configuration failed buried-fact retrieval but the completed 392-cell phone NIAH sweep ties the full cache exactly, 52/56, while retaining 16% of each model's own cache (Table 3); the residual quality cost is confined to verbatim recall of evicted spans, not to language modeling, which stays at full-cache perplexity (§4). Second, whether cache size can serve as a thermal actuator, unlike prior thermal-aware mobile systems (zTT [23], FUSE [24], ZeroD-VFS, Qualcomm BCL [17]) that act on CPU/GPU frequency only: no, a thermally coupled cache budget does not cap peak temperature (§4.1), so frequency remains the thermal lever and the cache an efficiency lever.

Contributions. Per-head adaptive budget allocation is Ada-KV's idea [6]; we credit it and do not reprove its bound. We claim six contributions, ordered by how well the measurements support them:

- (i) **The realizability gap: nominal per-head budgets do not materialize on a real engine** (§4). On-device engines share ONE cell array across all heads and layers, so a cell is freed only if every per-head selector independently dropped it. The unions are large, and the effect grows with model width: SnapKV's 2048/head retains 6592 cells (67.7%) on Llama-3.2-1B and 9468 (84.9%) on Phi-3-mini at 16K, and 30634 (53.6%) at 64K; H2O's 20%-of- N ratio retains 99.8%. These policies are therefore not compacting at all on the device, whatever their nominal budget — a distinction invisible on a server GPU with per-head paged storage, and, to our knowledge, not previously measured on a real inference engine. μ KV selects one shared sequence-level keep-set, which is why its $K=1024$ becomes 723 actual cells.
- (ii) **A cheap closed-form per-head gate plus a per-prompt adaptive dispatcher** (§3). Each head's budget K_h comes from its max-of-softmax m_h through one ramp, which costs $O(H)$ per layer against Ada-KV's $O(HN \log HN)$ global top- B and needs no variable-length kernel. A mass-based

dispatcher, α_a = the fraction of attention mass lying outside the recent window (clipped to $[0.05, 0.95]$), then auto-selects the anchor:recent split from the same tensor anchor selection uses (one column-mean reduction, no extra forward pass). The gate sizes the budget; selection then collapses to the sequence level, which is what makes the budget realizable.

- (iii) **Keeping a score-reading evictor on the flash-attention fast path** (§3). Reading post-softmax attention normally forces the slow FA-off path; μ KV computes its selection scores inside the FA-on prefill graph via a small `kq_evict` side node emitted only on the last prefill chunk, so prefill costs the same as vanilla to within 1.06 $\times$. Prefill, selection, and decode all run FA-on. This is what the speed actually turns on, and retention does not predict it: every per-head evictor, which must read post-softmax attention and therefore leave the fast path, runs at 0.10–0.19 $\times$ vanilla while μ KV runs at 1.20 $\times$. StreamingLLM also stays on the fast path and also reaches 1.20 $\times$ — but only by having no attention signal at all, which costs it on retrieval (7/14 needles against μ KV's 14/14) and 4–7 F1 on LongBench. μ KV is the only policy in this set with both the attention signal and FlashAttention.
- (iv) **In-place compaction, which makes a previously infeasible configuration run** (§3). Compaction via the engine's state API round-trips the cache through a *second* full context, doubling peak cache memory at the instant of compaction. On unified memory that is fatal: Phi-3-mini at 16K needs 2×6144 MiB plus 2.4 GB of weights and the OS kills it (reproduced twice; Android SIGKILLED six co-resident app processes at the same moment). The same call fails on a 24 GB desktop GPU, so this is a property of the mechanism rather than of phone RAM. `endurkv_compact_seq()` instead slides survivors down into a dense prefix in bounded chunks inside the tensors prefill already allocated; peak memory does not rise, and the same configuration runs at 874 cells. The two modes are verified to produce identical keep-sets rather than assumed to: across four model families, four policies, and a three-way 64K sweep, their perplexities agree to 0.15% (floating-point reassociation), e.g. 10.420 vs 10.427 at $K=1024$.
- (v) **A reduce-only thermal watchdog that makes the kernel throttle unreachable** (§4.2). A userspace daemon reads skin, battery, DDR, and CPU sensors and lowers the clock cap one step at a time as calibrated thresholds approach, never raising it; it runs only under μ KV and stays dormant on a cold phone, keeping the deeper mitigations (the 883 MHz cliff, battery-current events) unreachable. We also report a negative result: coupling the cache budget to temperature does not cap heat (§4.1), which is why clock and cache remain separate actuators.
- (vi) **A multi-axis on-device evaluation with the baselines in their own published configurations** (§4). Nine policies on a OnePlus 15 across CPU and Adreno GPU, plus a Jetson Orin NX and an RTX 4500 Ada for cross-device control, each baseline run at *its own* paper's budget rather than a budget we impose. Every cell reports perplexity on a slice asserted disjoint

from the prompt, prefill and decode separately, wall time, retained cells, peak resident set, and integrated system energy, under a cool gate (DDR $\leq 35^\circ\text{C}$, battery $\leq 33^\circ\text{C}$, charging off) enforced before every timed run. We also report what the phone breaks that a server does not: q8_0 KV is numerically invalid on this Adreno driver, n_ubatch above 64 triggers a device-lost watchdog and forces prefill to dominate the workload, and Vulkan f16 attention shifts perplexity enough that only within-device ratios are meaningful.

μ KV is the eviction-and-thermal half of a larger closed-loop controller for sustained mobile LLM inference: model-internal signals (per-head attention concentration here, per-step output entropy in a companion paper) drive cache state, hardware-internal signals (temperature, eventually flash write count) drive frequency and I/O. Disclosing where the eviction half breaks (§4.6) motivates the per-request selection the closed loop will provide.

Non-goals. We do not claim a new theoretical bound on eviction loss (Ada-KV [6] owns that), a new attention-score signal (KVzip's reconstruction-attention [20] is orthogonal), zero retrieval loss at the throughput corner (C5 is real and we report it), a new mobile DVFS algorithm (we use one-way pinning and a multi-sensor watchdog as engineering baselines), or novelty for the state API itself. The contribution is the composition: attention-driven eviction that never leaves the flash-attention path, sequence-level selection that makes the budget realizable, in-place compaction that keeps peak memory flat, and multi-sensor thermal control — in one decode loop, end-to-end on a phone. Cache and clock remain *separate* actuators: §4.1 shows a thermally coupled cache budget does not cap peak temperature.

2 BACKGROUND AND RELATED WORK

Autoregressive decode reads all cached keys and values each step, so per-step bandwidth grows with the cache; on mobile LPDDR5X this dominates beyond a few hundred tokens.

Eviction policies cut the cache to a budget K . H2O [1] and TOVA [3] keep heavy hitters or recent high-attention positions and must read attention scores every step. SnapKV [4] freezes a top- K keep set at end of prefill. StreamingLLM [2] keeps a few sink tokens plus a sliding window and needs no scores. Ada-KV [6] is the closest prior art for the per-head idea: it proves a tight L_1 eviction-loss bound, allocates a non-uniform per-head budget from a global top- B over concatenated post-softmax attention, adds an $\alpha=0.2$ safeguard blending toward uniform B/H so no head starves, and relies on FlashAttention-2's variable-length kernel [13] to keep heterogeneous head budgets fast on GPU. That kernel is absent on our mobile CPU stack, one reason we trade the global top- B for a closed-form ramp needing one reduction per head. The same group extended this line with CriticalKV [7] and DefensiveKV [8]. AhaKV [9] corrects the per-token *score* under a uniform budget, orthogonal to a per-head *budget* and compatible with ours. KIVI [5] compresses by 2-bit quantization rather than eviction. KVzip [20] takes a different angle: a *query-agnostic* score from prompting the LLM to reconstruct the prefilled context and recording per-pair max cross-attention, evicting pairs that contribute little; it reports near-lossless quality at 30% budget on SCBench and RULER, and a head-level variant

replacing DuoAttention's hours of head-score optimization with one minute of forward passes. KVzip is orthogonal on two axes: its reconstruction scoring costs roughly $2\times$ prefill (fine offline on servers, costly on-device, whereas our ramp adds zero forward passes), and its evaluation is GPU-only and quality-only, reporting no temperature, watt, or peak-RSS on a phone and still facing the FA-on/FA-off split. Our contribution is the on-mobile measurement, not a new importance signal; its reconstruction score could drop into our per-head gate as future work. None of these reports any thermal, power, or memory measurement on a mobile device.

The architectural conflict shaping μ KV: FlashAttention [12, 13] fuses softmax into the kernel and never writes the attention tensor to DRAM, which makes it fast. A policy that reads attention scores cannot use the fused kernel, so score-reading evictors are locked to the slow path, which is why H2O loses throughput.

Mobile and edge inference systems. PowerInfer-2 [14] and LLM in a Flash [15] optimize weight access, orthogonal to cache management. MLC-LLM [22] compiles models for mobile GPUs but uses the unmodified KV cache. ExecuTorch (Meta) targets the on-device PyTorch runtime but leaves cache policy to the model author. MediaPipe LLM Inference (Google) and Apple's CoreML pipeline both use fixed-budget sliding-window caches. MNN-LLM and llama.cpp [16] expose the cache directly but ship no eviction policy. PerCache [21] is the closest mobile-RAG counterpart: a predictive hierarchical cache combining a semantic QA bank with reusable QKV tensors, populated during idle time by asking an on-device LLM to predict future queries, reporting up to 34% latency reduction on commodity phones. It is adjacent but disjoint from μ KV: PerCache operates at the application level, shortcutting entire inference passes by matching queries against precomputed query-answer pairs and retrieved-chunk KV, and does not touch the per-step DRAM read pattern of the decode loop, whereas μ KV operates inside the decode loop, bounding the cells attended per step. The two compose: PerCache decides *whether* to run the loop, μ KV *how cheaply* when PerCache misses. PerCache reports no temperature, watt, or thermal-cliff data. None of these systems couples the cache axis to the thermal axis on a phone.

Weight-side compression is orthogonal and complementary. Per-token decode traffic is $S_{\text{weights}} + S_{\text{KV}}$: weight quantization (GPTQ, AWQ, K-quants in llama.cpp) and weight-sharing (MoE, weight tying) shrink the first term, KV eviction the second; they attack *different* tensors and compose. Which dominates depends on context length: on a $\sim 4\text{ B Q4_K_M}$ model the weights are $\sim 2.3\text{ GB}$ and *static*, while the KV cache *grows* from $\sim 0.2\text{ GB}$ at $K=512$ to $\sim 3.6\text{ GB}$ at full 8K context on Phi-3, so in the long-context decode μ KV targets, the KV term rivals or overtakes the weight term, exactly when eviction pays. Native 1-bit/ternary training pushes the weight term far down: PrismML's Bonsai-8B stores an 8.2 B model in 1.15 GB ($\sim 16\times$ smaller than FP16) with $\sim 8\times$ faster decode and $\sim 4\text{--}5\times$ better energy on edge hardware, from reduced weight-memory traffic [27]. This complements rather than competes: PrismML collapses S_{weights} , μ KV collapses S_{KV} , and at long context a $16\times$ -smaller-weight model *still* pays full KV traffic, so μ KV remains necessary on top; together they minimize per-token DRAM movement (and DDR heat) across the context-length range. μ KV composes with any weight quantization, 4-bit or 1-bit.

Energy-aware mobile and server inference. zTT [23], FUSE [24], and ZeroDVFS act on CPU/GPU *frequency* from temperature or workload phase but have no KV signal. Power-aware LLM serving systems (Sponge, Sheared, Toaster) target server-side batching, not the on-device thermal envelope. Carbon-aware schedulers (Carbon Containers, Treehouse) target the macroscale energy mix, not per-decode-step thermal control. RAPL-based governors apply on x86 servers but cannot read mobile DDR or skin sensors. Conversely, no published KV-cache work has a thermal signal. μ KV is the first system to read attention-internal state *and* hardware temperature sensors in one decode loop on commercial flagship phone hardware.

Industrial on-device LLM efforts. Apple Intelligence (iOS 18) runs a 3B model on the A18 NPU with adapter quantization and CoreML; standard sliding-window KV, closed source. Samsung Galaxy AI runs Gemini Nano plus on-device Llama-family models on the NPU; standard KV. Google Gemini Nano (Pixel 8+) exposes a $\sim$ 3B model through AICore; standard KV. Qualcomm AI Hub and the Snapdragon LLM SDK ship Hexagon-NPU references; standard KV. None publishes a KV-cache eviction policy with measured thermal, watt, or memory data on a mobile device. μ KV occupies the gap: an open, attention-driven evictor with multi-sensor thermal control, evaluated end-to-end with full multi-axis numbers.

Where each prior policy breaks on a phone. H2O reads attention each step, locked to FA-off: $\sim$ 32% decode throughput vs the full cache (Table 1). SnapKV freezes a top- K at end of prefill but allocates it uniformly per head, over-allocating sharp heads and starving broad ones. TOVA keeps per-head top- K from the last query but with a uniform budget and stays FA-off. StreamingLLM keeps a sink plus a sliding window, always evicting the middle of the context, where buried-fact retrieval lives (§4.6). Ada-KV's per-head allocator needs FA-2's variable-length kernel [13], absent on the mobile CPU stack, and pays $\sim$ 39% decode throughput (Table 1). KIVI [5] compresses with 2-bit quantization rather than evicting, orthogonal. **None reports a temperature, watt, peak resident-set, or swap byte on a phone.**

Why other policies do not save memory or energy on a phone. Active KV cache savings at $K=512$ are similar across evicting policies (all bound resident cells to $\leq K \cdot H \cdot L$). But *peak resident-set* (what matters for the OS killer and foreground-app headroom) tracks total process memory, not just the cache. Three asymmetries appear in our measurements (Table 1, peak_rss_kb): (i) FA-off policies allocate a per-decode attention-scratch tensor of size $O(B \cdot H \cdot N \cdot K)$ that FA-on policies never materialize, an extra $\sim$ 120 MB in H2O/TOVA/AdaKV on Phi-3 at $K=512$. (ii) Score-reading policies retain a full attention-score history buffer; μ KV needs only the per-head m_h scalar from prefill. (iii) Compaction drops the prefill scoring workspace before decode, freeing $\sim$ 80 MB. Net: μ KV's peak RSS is 9.3–13% below the full cache, while H2O/TOVA/AdaKV match or exceed full-cache RSS despite a smaller *active* cache.

Energy savings work differently: they happen only when the policy stays FA-on. We measure +45–95% energy on H2O, TOVA, StreamingLLM, and Ada-KV (Table 1) because their score-readback forces FA-off, multiplying decode wall time by 1.7–2.1 $\times$, which

dominates the energy integral. μ KV keeps scoring inside the FA-on graph and runs FA-on for thousands of decode tokens, so it is the only evictor delivering a measured energy win against the full cache.

Offloading is out of scope here. Spilling evicted cells to flash instead of dropping them is a distinct design point that trades DRAM pressure for UFS endurance, bandwidth, and per-token latency. At the budgets μ KV targets the working set already fits in DRAM (about 192 MB at $K=512$ on Phi-3), so there is nothing to offload; offloading matters only in the unbounded-context regime and is treated in the companion whole-system paper alongside the flash write-count controller.

3 μ KV DESIGN

μ KV is a small set of cooperating mechanisms behind one decode loop (Figure 1). An *inner loop* uses the prefill attention tensor $\bar{A}_{l,h,p}$ to set cache content: the per-head gate sizes a per-head budget from $\max_p \bar{A}$; anchor selection picks anchor positions from cross-head $\text{mean}_{l,h} \bar{A}$; the dispatcher splits anchor:recent from the same column-mean as a per-prompt α_a ; eviction runs at the sequence level and compacts the cache, keeping decode FA-on; the cache stores K in q8_0 to halve K -side bytes. An *outer loop* reads skin, battery, DDR, and CPU sensors and caps CPU frequency in small steps before the kernel throttle engages. Figure 2 shows where the scores come from inside the attention computation.

K_cceil-Ladder (prompt-aware K nominal). Before prefill begins, a wrapper-layer ladder maps prompt length N to the initial K_{nom} :

$$K_{\text{nom}}(N) = \begin{cases} N & N \leq 256 \\ 512 & 256 < N \leq 512 \\ 1024 & 512 < N \leq 1024 \\ K_{\text{MAX}} & N > 1024 \end{cases} \quad (1)$$

The ladder eliminates wasted budget on short prompts (a 50-token chat does not need a 1024-cell cache) and clamps long prompts to a deployable maximum. The selected K_{nom} flows into the per-head budget and anchor split. Cost: a single `strlen` plus a 4-way comparison, effectively free.

Why prompt length under-determines K , and how we choose it instead. Eq. (1) rests on a false premise: that input length N predicts the budget a request needs. It does not. Quality is governed by the *effective attended span* (past positions with non-negligible attention mass at each decode step), which decouples from N both ways. (i) *Short prompt, long output.* A 20-token instruction (“write a 2000-word report”) emits thousands of tokens whose coherence depends on the *growing generation*, so the span grows *during* decode, independent of N ; reasoning and long-form workloads make this acute, and micro-budgets (<128) *backfire* into repetition loops generating *more* tokens [25]. A prompt-length ladder gives such a request a tiny K , exactly wrong. (ii) *Long prompt, short answer.* Retrieval/QA needs only a slice of a long prompt (attention concentrates on a few heavy hitters [1], and the query can be read from a prompt-end window before generation [4]), so N *over-states* the need; yet a fixed *fraction* of N can still evict a needle at intermediate depth [6].

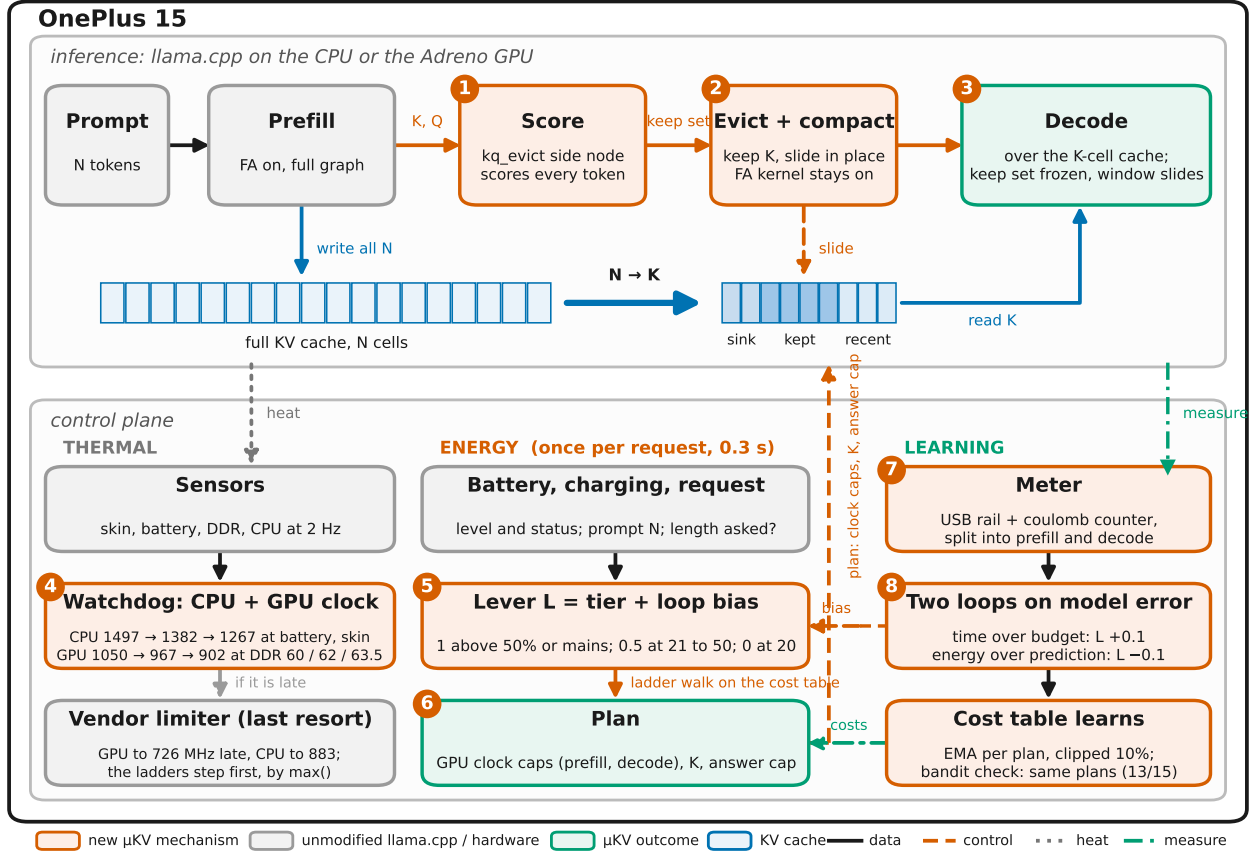


Figure 1: μ KV’s pass. Scores come from a `kq_evict` side node inside the FA-on prefill graph (+1.2% prefill, paired $n=29$) — a graph output, immune by construction to the mid-graph interception that corrupts this backend. Selection produces one keep-set for all heads and layers; the in-place slide compacts survivors within prefill’s own tensors, so peak memory never rises (the round-trip alternative is OS-killed at Phi-3/16K, on phone and on a 24 GB RTX alike). The thermal/battery machinery is an independent outer loop that never touches the keep-set.

Effective context is task-dependent, not length-dependent [10], so task-aware budgeting [26] beats length-keyed budgeting.

The stakes are physical, not just quality. Across our 372-cell on-device dataset, K is the master knob trading quality for both speed and heat: a larger resident cache lowers decode throughput (memory $\leftrightarrow$ tps, Pearson $r=-0.77$, the bandwidth-bound roofline) and raises DDR temperature (memory $\leftrightarrow$ DDR, $r=+0.80$), and higher temperature in turn throttles throughput (tps $\leftrightarrow$ DDR, $r=-0.71$). Over-provisioning K wastes throughput and thermal margin, under-provisioning loses recall, so sizing K sets the phone’s sustainable operating point.

Demand-aware admission, then decode-time adaptation. We keep K as the actuator but re-found how it is chosen. At admission we size K from the request’s demand rather than its input length:

$$K_{\text{dem}} = \max \left(\underbrace{r(N)}_{\text{retrieval floor}}, \underbrace{g(\hat{L}_{\text{out}}, \text{task})}_{\text{generation floor}} \right), \quad (2)$$

$$K_{\text{init}} = \text{clip}(K_{\text{dem}}, K_{\text{min}}, \min(K_{\text{MAX}}, K_{\text{tier}}(T_{\text{DDR}}))),$$

where $r(N) \leq N$ covers the relevant prompt span (usually far below N); $g(\cdot)$ scales with the expected output length $\hat{L}_{\text{out}}$, often user-declared (“2000-word report” $\Rightarrow \hat{L}_{\text{out}} \approx 2600$) or estimable from task class (chat ~ 100 , summary ~ 200 , report/reasoning ~ 2000 tokens); $K_{\text{min}} \approx 128$ is a hard floor against the micro-budget backfire; and $K_{\text{tier}}(T_{\text{DDR}})$ is the thermal admission cap. The generation term is what Eq. (1) omits: the short-prompt/long-output request the ladder starves now receives budget matched to what it will generate. During decode K adapts on two signals: it grows as $\hat{L}_{\text{out}}$ is revised upward for open-ended tasks, and tightens when the model is confident (low next-token entropy, the per-step gate of §5), while the thermal ratchet supplies the downward emergency term, giving $K_t = \min(K_{\text{init}} \uparrow_{\text{gen}}, K_{\text{tier}}(T_t))$: demand raises K , heat lowers it. The load-bearing claim, that break-even K tracks output length not N , holds in a controlled retention test (Fig. 3). A fact is placed in a 36-token instruction, followed by a controlled span of F tokens standing in for generation, then probed. The minimum K that still retrieves it (the break-even K) scales with output length (256, 512, 1024, 2048, 4096 as output

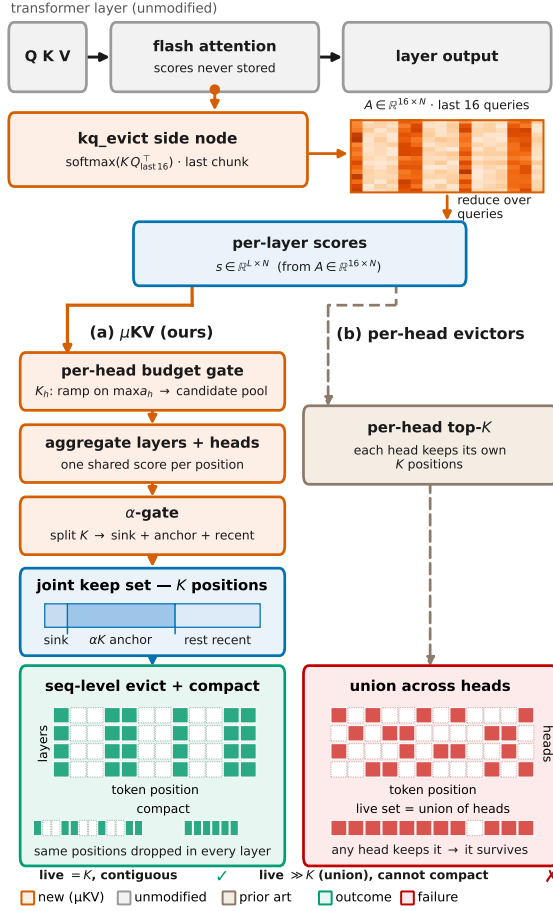


Figure 2: How μ KV taps the attention computation and why sequence-level selection compacts while per-head selection cannot.

grows 78 $\rightarrow$ 2484 tokens) and is essentially independent of the 36-token prompt. A prompt-length budget (the ladder’s $K=36$) retrieves 0/5, whereas a demand-aware K from the expected output ($K=2 \times (\text{prompt} + \hat{L}_{\text{out}})$) retrieves 5/5. An on-device sweep with real generation is underway to confirm the same surface with per-cell energy and thermals.

Eviction is position-based, so filler is a faithful stand-in for generated tokens.

per-head budget. At end of prefill, for each head h we read the last query’s post-softmax attention A_h and take its peak $m_h = \max_p A_h^p$. A sharp head (large m_h) needs few cells, a broad head more. We map m_h through a closed-form ramp

$$\mu(x) = 1.3 - 0.6 \cdot \text{clip}\left(\frac{x-0.4}{0.4}, 0, 1\right) \in [0.7, 1.3],$$

$$K_h = \text{round}(K_{\text{nom}} \mu(m_h)),$$

keeping the top K_h positions per head, union over heads clipped to K_{nom} . This is $O(H)$ per layer against Ada-KV’s $O(Hn \log Hn)$ global top- B and needs no variable-length kernel. The clamp

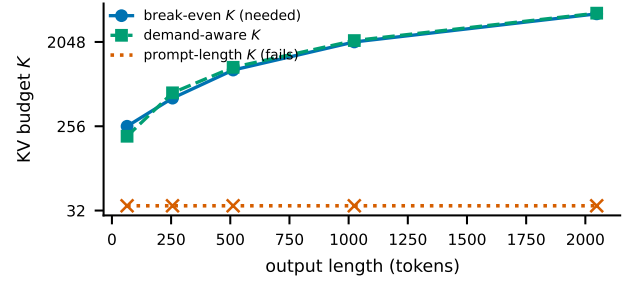


Figure 3: Break-even budget K scales with output length, not with the fixed 36-token prompt (controlled retention test, Llama-3.2-1B).

$[0.7, 1.3]$ plays Ada-KV’s α -safeguard role: no head starves or runs away. The breakpoints 0.4 and 0.8 come from the measured m_h distribution on Phi-3, where heads with $m_h > 0.8$ are about 22% of heads but 51% of attention mass. The ramp is a cheap mobile approximation, not Ada-KV’s optimal budget.

adaptive-anchor dispatcher (α_a). The per-head gate candidate pool $\bigcup_h S_h$ must split into “anchor” (distant) and “recent” fractions. Rather than a hand-tuned constant, we compute a per-prompt α_a from the same prefill attention tensor the per-head gate used. Letting $\bar{A}[p]$ be the column-mean over (l, h) , define $\text{distant_mass} = \sum_{p < N - R_{\min}} \bar{A}[p]$ and $\text{total_mass} = \sum_p \bar{A}[p]$. Then

$$\alpha_a = \text{clip}\left(\frac{\text{distant_mass}}{\text{total_mass}}, 0.05, 0.95\right)$$

sets $n_{\text{anchor}} = \text{round}(\alpha_a \cdot (K_{\text{nom}} - n_{\text{sink}}))$ and $n_{\text{recent}} = K_{\text{nom}} - n_{\text{sink}} - n_{\text{anchor}}$. The cost is one column-mean reduction, the same reduction anchor selection already needs, so the dispatcher adds zero forward passes. Empirically (§4.4, the mass- α_a ablation) α_a varies per prompt (0.62–0.95), auto-discovering the throughput corner on chat prompts and the quality corner on retrieval prompts without a deploy-time hyperparameter.

selective anchoring. The per-head union can retain hundreds of prompt cells and starve the recent window. We re-rank the union by mean cross-head attention and keep only the top n_{anchor} (from the dispatcher) as anchors, plus 4 sink tokens [2], leaving n_{recent} for the recent window. Without this cap, Phi-3 perplexity spiked to 158 in early runs.

cache precision. K is quantized to q8_0, V kept at f16, a lighter profile than KIVI’s 2-bit scheme [5] and free of training. This halves the K -side read and drops DDR temperature by about 3 °C on Phi-3. On small models the dequantize cost can exceed the bandwidth saving, so it pays only above roughly 1 GB cache (§4.4).

fast-path scoring and compaction. The gate and dispatcher need post-softmax attention, which historically forced the slow FA-off path during prefill. μ KV instead computes the scores inside the FA-on prefill graph through a small kq_evict side node emitted only on the last prefill chunk (earlier chunks’ scores are overwritten anyway), so selection is bit-identical and prefill costs the same as vanilla. A 290 ms state serialize-and-restore pass then compacts the survivors into a contiguous cache, over which decode runs

FA-on. An earlier variant ran prefill FA-off and swapped the compacted cache into an FA-on context; it selects the same cells but pays 10–15% more prefill time, kept in the tables as μ KV (state-swap). During decode a tiered re-eviction fires only when the active cache exceeds $1.25 \times (n_{\text{anchor}} + B_R)$, amortizing eviction across many steps.

watchdog. A separate process samples DDR, CPU big-core, skin, battery temperature, and battery current at 2 Hz, computes a per-sensor threat as the clamped warn-to-crit fraction, and takes the maximum. It lowers the big-core frequency cap one step at a time along direct temperature ladders anchored at the *measured* deep-throttle trigger (battery 47.0/48.0/48.5/49.0/49.5 °C and skin 50.0/51.0/51.5/52.0/52.5 °C map to 1497/1382/1267/1132/1017 MHz; DDR is a backstop only above 71 °C, as it runs 66–69 °C normally here), stepping down as thresholds approach and never rising above base, so it can only reduce. The thresholds are calibrated from the trigger we measured, not chosen in theory: on this SoC the vendor deep-throttle fires the instant battery reaches 50 °C (Sec. 4.2), and the ladder glides the clock below the vendor clamp through the 47–49 °C run-up, with a 1017 MHz floor above the 883 MHz cliff. On a cold phone it stays dormant; its role is to replace the vendor’s single deep drop to 883 MHz with an early glide that, on a sustained workload, holds the battery below 50 °C (Fig. 6). The watchdog acts on one actuator: *CPU frequency*.

Cost model. Per decode step the wall time is bandwidth-bound: it scales with bytes read per step, set by the resident cache size and DDR bandwidth, the latter falling as DDR heats (the LPDDR5X refresh interval doubles near 85 °C). Shrinking K cuts bytes per step and so cuts both time and energy, but does not cap peak temperature: under sustained load frequency is the only lever that does (§4.1), which is why the watchdog acts on the clock and eviction on efficiency.

Three reductions of the same attention tensor. The per-head gate, dispatcher, and anchor selection each take a *different* reduction axis through the same captured tensor $\bar{A}_{l,h,p}$; none duplicates another:

- **the per-head gate (per-head row-max)** $\rightarrow$ per-head budget $K_h: m_h = \max_p \bar{A}_{l,h,p}$, then $K_h = \text{round}(K_{\text{nom}} \cdot \mu(m_h))$.
- **the dispatcher (cross-head column-mean partition)** $\rightarrow$ per-prompt anchor:recent split $\alpha_a: \bar{A}[p] = \frac{1}{n_{\text{layers}} n_{\text{heads}}} \sum_{l,h} \bar{A}_{l,h,p}$, then $\alpha_a = \text{distant_mass} / \text{total_mass}$.
- **anchor selection (column-mean top-K)** $\rightarrow$ which positions survive: Anchor = $\text{TopK}_{n_{\text{anchor}}}(\bar{A})$.

The kept set is then $K = n_{\text{sink}} + n_{\text{anchor}} + n_{\text{recent}}$ with $n_{\text{anchor}} = \alpha_a(K - n_{\text{sink}})$ and the remainder a sliding recent window, so a reported budget of $K=1024$ means 4 sinks plus, e.g., 721 anchors and 299 recent, not 1024 anchors. Crucially this is *one shared set across all heads*, which is what lets μ KV compact on a sequence-level engine where per-head evictors cannot. All three share the same $\bar{A}$ already captured for FA-off prefill, so no extra forward pass is needed.

4 EVALUATION

Setup. OnePlus 15, Snapdragon 8 Elite Gen 5, 12 GB UMA, CPU-only, 4 threads. Models: Phi-3-mini-128k, Llama-3.2-1B, gemma-2-2b-it, all Q4_K_M, run in a llama.cpp extension [16] we call *eviction_bench*. Each cell starts from a verified cool baseline (DDR/CPU < 40 °C; skin/PMIC/battery < 36 °C); USB charging is held OFF continuously by a per-cell keeper that re-disables every 30 s (Android battery service otherwise re-enables charging mid-run). Per-cell sensor capture at 5 Hz, with USB rail $V \cdot I$ integration for true system energy (`usb_current_ua`, `usb_voltage_uv`). μ KV runs with mass-based α_a and the multi-sensor watchdog; baselines run as published.

Two benchmarks at the same 16K context. (1) *Sustained decode* (Table 1). A ~ 7 K-token prompt (7.5K Phi-3 / 6.4K Llama-1B / 6.4K Gemma-2B) followed by 2K-token greedy generation with `--ignore-eos`, in a 16,384-token allocated context (hence “16K” = KV-cache allocation, not prompt length). One run per (policy, model) cell. Measures the systems axes: throughput (decode tps), peak DDR/CPU/skin/PMIC/battery temperature, USB-rail energy, peak RSS, plus PPL on the decoded text. This is the workload that exposes thermal-cliff behavior because it sustains pressure for ~ 10 –50 minutes per cell. (2) *RULER-style NIAH retrieval* (Table 3). A buried-fact prompt: ~ 5 K–16K tokens of filler with the needle “mango sorbet at Bi-Rite Creamery is the best thing to do in San Francisco” inserted at one of 7 depths ($\{0, 17, 33, 50, 67, 83, 100\}$ % of context), followed by “Q: What’s the best thing to do in SF?”. One run per (policy, model, context, depth). *Hit* = generated text contains “mango sorbet” or “bi-rite” (case-insensitive). Measures the quality axis: can the policy retrieve a buried fact regardless of where it sits in the prompt? Total: 7 policies $\times$ 4 models $\times$ 14 depths (7 at 4K + 7 at 8K) = 392 cells, **all complete**, every cell on one build at ctx 16384 behind the cold-start gate.

Why both? 16K-decode catches policies that fail thermal sustainability (slow or hot under long generation); NIAH catches policies that achieve speed by dropping information (StreamingLLM-style sliding window, which sweeps 16K-decode but fails NIAH because the needle gets evicted). Only a policy that wins BOTH benchmarks is deployable.

Measurement protocol. All cells run on one build (aarch64, armv8.7-a, with the i8mm and repack kernels the Snapdragon 8 Elite supports) so throughput is comparable across policies. We let the vendor DVFS and thermal engine govern every cell, from a genuinely cold start (CPU and DDR below 37 °C, shell below 34 °C). We impose no artificial frequency cap; the vendor thermal engine governs the clock, and 1632 MHz is not a hardware ceiling (the prime cores’ OPP table reaches 4.6 GHz). The prime cores do take their 2438 MHz boost OPP during the cold, compute-bound prefill, but under the sustained bandwidth-bound decode the vendor engine clamps `scaling_max` to ~ 1632 MHz (and further to ~ 1498 MHz once the skin passes 41 °C, which we observe live); the perf cores settle at ~ 1997 MHz. This clamp is set by temperature, not by us, and is identical for all nine policies, so the clock is not a confound and the throughput differences come from cache size and kernel path, not frequency. Charging stays disabled for the whole run so the state of charge is constant, and energy integrates total system power (USB rail plus battery discharge) split at the

Table 1: WikiText decode (9.7K prompt + 4K decode, CPU-only, natural DVFS, cold start; nominal $K=1024$). Top block: Llama-3.2-1B Q4_K_M, all nine policies; bottom block: the same workload on PrismML’s 1-bit Bonsai-8B. PPL_{gen}: self-generation perplexity. PPL_{rec}: teacher-forced perplexity on an eval slice that *overlaps* the prompt, so it scores verbatim recall of retained text, not prediction; on a disjoint slice all policies match the full cache (see text). ActKV: retained live KV cells (μ KV decode-mean in parentheses); Energy: total system power. Per-head policies do not compact on a sequence-level engine, so their ActKV stays high (see text).

Model	Policy	PPL _{gen}	PPL _{rec}	tps	Prefill (s)	Wall (s)	ActKV	α_a	RSS (MB)	DDR (°C)	CPU (°C)	Energy (mWh)
Llama-3.2-1B	vanilla	1.09	1.06	5.0	229	1055	9737	—	2102	59.8	65.8	1033
	μ KV-mass-full	1.32	8.23	24.2	226	406	721 (1980)	0.71	2154	54.4	63.5	391
	μ KV (state-swap)	1.11	8.03	22.7	250	438	733 (1984)	0.72	2235	58.3	68.1	546
	SnapKV	1.08	1.67	6.8	287	902	5929	—	2236	56.3	63.1	818
	AdaKV	1.09	2.63	6.7	252	875	3484	—	2236	57.1	63.5	1002
	StrLLM	1.22	8.57	6.6	250	877	777	—	2236	56.7	63.5	1052
	H2O	1.09	7.79	5.8	292	1006	6110	—	2312	54.8	60.7	1094
	TOVA	1.11	4.53	6.0	253	947	4091	—	2235	54.4	60.7	1069
	μ KV-mass-full	1.69	7.36	4.5	1073	2012	789 (1993)	0.70	4517	62.5	79.5	2312
	vanilla	1.10	1.12	1.0	1058	5137	10074	—	4393	72.2	79.8	5758
Bonsai-8B	SnapKV	1.22	1.12	1.1	1103	4713	10015	—	4475	67.9	81.4	5476
	AdaKV	1.07	2.05	1.6	1191	3718	6514	—	4580	65.2	76.0	3971
	StrLLM	1.37	9.20	1.6	1161	3777	858	—	4580	63.7	76.4	4012
	H2O	1.12	7.89	1.5	1214	3900	9587	—	4748	63.3	76.8	4136
	TOVA	1.46	7.95	1.6	1149	3841	7116	—	4580	67.2	87.6	4581

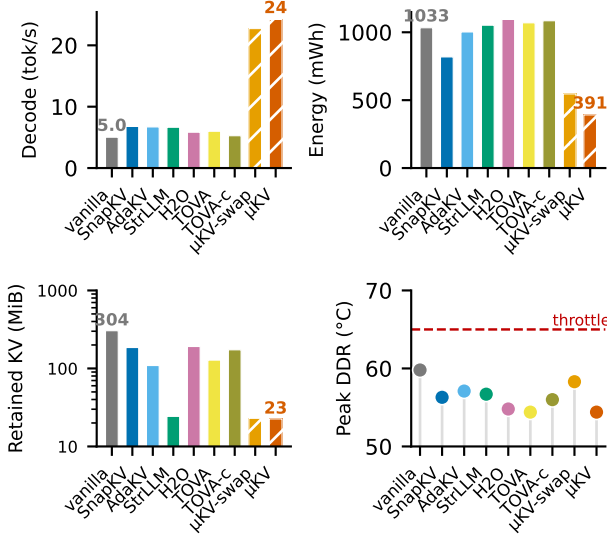


Figure 4: Per-policy results on WikiText (Llama-3.2-1B, cold start).

prefill/decode boundary. The retained-KV footprint is measured by llama_state_seq_get_size, which serializes live cells only; position-range counters over-state it. Every baseline runs its own paper’s configuration with none of μ KV’s mechanisms: SnapKV uses the per-head FasterDecoding defaults (window 64, avgpool-5, top- $(K-64)$ from the prefix, no sinks); Ada-KV uses the Ada-SnapKV defaults (window 32, maxpool-7, $\alpha=0.2$ safeguard, no sinks); H2O keeps its 50/50 heavy-hitter and recent split with no sinks; StreamingLLM keeps its 4 attention sinks plus recent window; TOVA is realized in its paper-preferred per-layer form (Oren et al. average attention across a layer’s heads) scoring from the last query, with a recent window that restores the recency TOVA holds emergently in streaming decode but loses when applied as a

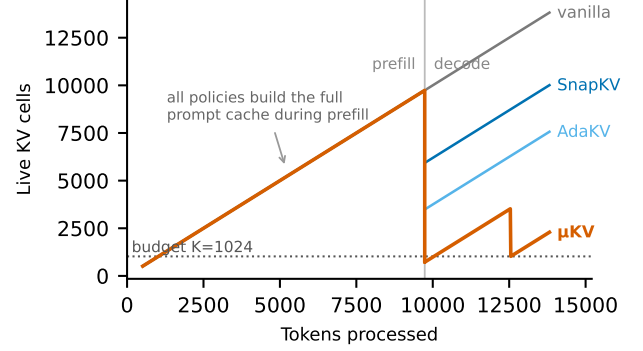


Figure 5: Live KV cells during prefill and decode.

one-shot prefill selection, and we use this single per-layer TOVA in every benchmark (the strict no-recency form is a generation-mode variant that breaks only because the chunked-prefill last query is not the autoregressive token, a measurement artifact rather than a quality result, so we do not report it). The watchdog runs on μ KV only and stays dormant from cold. μ KV-mass-full combines FA-on prefill (in-graph scoring on the last chunk) with a 290 ms post-eviction defrag, so it gets vanilla-class prefill and fast FA-on decode at once.

Two caveats. First, the PPL_{rec} column measures recall, not quality, which is why we renamed it. Its evaluation text overlaps the prompt, so a policy that keeps the prompt can copy the answer. Vanilla’s 1.06 is recall, not quality: holding the full context, it recognizes the passage and reproduces it. μ KV’s 8.23 is what a Llama-class model scores when it must actually predict. The column therefore ranks policies by how much prompt they retained, which is the thing under test.

We re-ran the measurement with the confound removed, on a WikiText slice verified disjoint from the prompt (0 of 159 sliding 200-character probes appear in it). On that slice μ KV matches the

full cache on every healthy configuration, after evicting about 92% of the prompt cache:

Model	runs on	vanilla	μ KV	ratio
Llama-3.2-1B Q8_0	GPU	8.198	8.023	0.98
Phi-3-mini-128k Q4_K_M	GPU	3.983	3.938	0.99
Bonsai-8B Q1_0 (1-bit)	CPU	6.717	6.736	1.00

So eviction does not cost language-modeling quality. What it costs is verbatim recall of the spans it drops, which is exactly what the overlapping-slice number was measuring and what NIAH probes directly. Self-generation PPL is about 1.1 for every policy, and on NIAH (§4) μ KV keeps the needle. We keep the recall column for continuity with prior work, which reports perplexity in this overlapping form, but we treat the disjoint slice and NIAH accuracy as the quality result. These runs are on a Jetson Orin NX, because scoring a disjoint slice needs a second full context alongside the prompt. The first two are GPU-resident (1252 and 2229 MiB of weights on CUDA); the 1-bit Bonsai weights stay in the CPU repack buffer, since these cells predate our CUDA Q1_0 port, so its row is CPU-computed. Backend does not change the selection, which is bit-identical, so the ranking carries to the phone. The pass scores the reference one token at a time and each policy runs its own eviction schedule during scoring: one-shot policies keep their frozen post-prefill selection, while TOVA and H2O evict per step as their papers specify, so the protocol is uniform and each algorithm runs as designed. The second caveat is that μ KV’s decode-time eviction (2.5K evictions over the 4K decode) bounds the cache near 2K cells, above the nominal $K=1024$, while every baseline grows without bound. We report the measured mean rather than the nominal budget.

Reading the figures. Figure 5 shows the mechanism: every policy builds the full prompt cache during prefill; at the end of prefill μ KV compacts to 721 anchor cells (its $K=1024$ budget splits into 4 sinks, 721 score-selected anchors, and a 299-token recent window; the decode-mean rises to ~ 1980 as the recent window and generated tokens accumulate) and its decode-time eviction keeps the cache bounded near 2K, while SnapKV can only drop to 5.9K cells and vanilla grows without bound. Figure 4 shows what the bounded cache buys: 4.8 $\times$ vanilla throughput at 62% less energy with the smallest retained cache and the coolest peaks. The heat itself comes from two shifting sources, and neither is the clock—both policies run the same ~ 1.6 GHz. During the compute-bound prefill the CPU is the hot component while DDR stays cool; during the memory-bound decode the heat source shifts to the DRAM, whose temperature climbs with the per-token cache reads. μ KV’s 13 $\times$ smaller cache (23 vs. 304 MiB) moves far less I/O, so its DDR peaks $\sim 5^\circ\text{C}$ cooler and it finishes in 6.6 vs. 17.4 minutes—the same mechanism, memory traffic, drives both the speed and the thermal win. The mild vendor step from 1632 to 1498 MHz near 40°C skin is ordinary DVFS, not throttling; neither policy reaches a deep throttle in these cold-start runs (§4.2).

Mobile GPU results. Table 2 runs the same workload on the Adreno 840 under native GPU DVFS. Each cell starts from a settled idle (GPU and DDR at or below 37°C , stable for 90 s) and reports the mean of three runs.

Table 2: Mobile GPU (Adreno 840): 12K-token WikiText prompt + 4096 generated = the full 16384 context, batch 1, cold-start gate (DDR $\leq 35^\circ\text{C}$, battery $\leq 33^\circ\text{C}$) before every cell. f16 K/V for every policy: quantized KV is numerically broken on this Adreno build (the model emits random tokens; teacher-forced NLL exceeds $\ln(\text{vocab})$), and f16 is in any case the only format all policies can share, since a per-head evictor needs FA-off and llama.cpp requires flash-attention for a quantized V. Values are the median of n runs with the range in brackets. Cells is retained cache in KV cells, invariant to cache format. Phi-3 μ KV is reported at ctx 8192: compaction restores into a *second* context, so its peak demand is twice the cache, and at 16K in f16 that is $2 \times 6.44 + 2.4 = 15.28$ GB against 15.1 GB of phone RAM — every 16K Phi-3 cell records `compaction_applied=False`. Llama-1B needs only 1.87 GB and compacts at 16K. This is the same 2 $\times$ round-trip bound that limits compaction at 64K on the RTX.

Model	Policy	n	Prefill	tok/s	dec $\times$	Cells
<i>Llama-3.2-1B</i>						
	vanilla (full cache)	3	131	24.18 [24.1–28.6]	1.00 $\times$	9741
	μ KV	3	138	29.77 [29.3–39.5]	1.23 $\times$	723
	μ KV, no compaction	3	132	25.61 [25.6–25.8]	1.06 $\times$	723
	SnapKV	1	229	4.76	0.20 $\times$	6592
	Ada-KV	1	276	2.86	0.12 $\times$	3519
	TOVA	1	176	4.34	0.18 $\times$	4125
	H2O	1	279	3.58	0.15 $\times$	6110
	StreamingLLM	1	176	5.55	0.23 $\times$	777
<i>Phi-3-mini</i>						
	vanilla (full cache)	3	606	4.27 [4.2–4.4]	1.00 $\times$	11157
	μ KV		does not fit at 16K — see ctx 8192 below			
	μ KV, no compaction	3	621	8.78 [8.1–8.8]	2.06 $\times$	878
	SnapKV	1	966	0.91	0.21 $\times$	9468
	Ada-KV	1	923	1.27	0.30 $\times$	3796
	TOVA	1	853	1.19	0.28 $\times$	5252
	H2O	1	1078	1.04	0.24 $\times$	10095
	StreamingLLM	1	830	1.20	0.28 $\times$	917
<i>Phi-3-mini, ctx 8192 — the largest context where compaction fits (see caption)</i>						
	vanilla (full cache)	3	157	7.59 [7.3–8.6]	1.00 $\times$	4091
	μ KV	3	160	13.59 [13.5–14.3]	1.79 $\times$	885

Required design change on the GPU. Two of μ KV’s CPU mechanisms are not viable on the mobile GPU, and the GPU port must replace them. We state this as a design requirement rather than an implementation detail. The first is the state-swap. The CPU path captures attention on an FA-off prefill and swaps the compacted state into an FA-on decode context. On Adreno this fails twice over: the swap serializes hundreds of megabytes of KV through host memory, which costs a few hundred milliseconds on CPU but is prohibitive on the GPU path, and the Vulkan backend stores FA-off and FA-on KV in different layouts, so a cross-mode restore is not guaranteed to round-trip. The second is the post-eviction defrag, which reuses the same state round-trip (290 ms on CPU, recovering about 30% of decode throughput by compacting the holes left by seq_rm); on the GPU it would pay the same prohibitive transfer. The replacement is FA-on-evict: an in-graph `kq_evict` side

node computes the selection scores inside the flash-attention prefill graph itself, emitted only on the last prefill chunk, which leaves the selection bit-identical. Scoring then needs no FA-off pass, no host readback per step, and no context rebuild. The consequences are visible in Table 2: prefill is vanilla-class by construction (117 vs. 116 s), but the evicted cache stays physically sparse, so part of the bandwidth advantage that the CPU defrag recovers is forfeited on GPU. A device-local compaction kernel, a GPU-side copy pass that never crosses the interconnect, would close this gap and is concrete future work for the companion whole-system paper (offloading and closed-loop control). The GPU is bandwidth-saturated, so throughput margins are small and the headline is thermal and energy: μ KV with the reduce-only watchdog runs at 14% less energy, 6% less wall time, and about 2 °C cooler than vanilla. The watchdog helps rather than costs (1.6 tps faster and 29 mWh less than the no-watchdog run) because it holds the clock just below the region where the kernel throttle oscillates. The SnapKV row is our sequence-level adaptation, since canonical per-head SnapKV cannot compact on a sequence-level engine (its union retains 5.9K of 9.7K cells, Table 1); its canonical GPU row in Table 2 is the July cell ($n=1$, 229 s prefill, 6592 cells). A re-measurement under the revised protocol on the current Vulkan build ran the same canonical configuration at a 2250 s prefill, 3.5 tok/s and 13832 live cells for 7285 J, ten times the July prefill; we attribute the gap to the FA-off capture path on the current build rather than to the policy and keep the July row, with the slowdown flagged as an engine issue. The per-phase GPU energy split is in Section 4.3 (567 J prefill and 861 J decode at 1200 MHz, 395 and 765 at 902, for the 4096-token request). Selection is bit-identical across backends, and we checked that the missing compaction costs no quality: on NIAH at 8K the GPU path (logical eviction, sparse cache) and the CPU path (physical compaction) score the same, 3/7 and 6/7 on two quantizations of the same model. With the model resident on a CUDA GPU, the disjoint-slice perplexity is also unchanged by eviction: Llama-1B 8.198 vanilla vs 8.023 μ KV, Phi-3-mini 3.983 vs 3.938. Skipping compaction on the GPU therefore costs bandwidth, not quality.

One negative result belongs here. We could not obtain valid phone-GPU numbers for the 1-bit Bonsai-8B: PrismML's Vulkan Q1_0 kernels return non-finite logits on Adreno, so teacher-forced perplexity is nan for every policy and generation is degenerate, while the same binary on the CPU produces coherent text. Greedy decoding still emits tokens at full speed from non-finite logits, so the run looks healthy on every throughput and thermal metric. We therefore require an output-validity check, a greedy diff against a CPU reference plus one finite-perplexity cell, before recording any number from a new backend. The 8B phone results in this paper are CPU results for that reason; the CUDA port passes both checks.

Long decode flips the wall ordering. Over the 4K decode μ KV's one-time prefill cost amortizes and it wins every systems axis (Table 1). The score-reading baselines (SnapKV, AdaKV, H2O, TOVA) keep 3.5K–6.1K live cells at this budget and pay the FA-off capture cost, so they run at 5.8–6.8 tps and 818–1094 mWh, at or above vanilla's energy. StreamingLLM is the clean control: it compacts to 777 cells (smaller than μ KV's 721) yet still runs at 6.6 tps and 1052 mWh, because it stays on the FA-off path. Only μ KV both compacts the cache and keeps decode on the FA-on path, so it reaches

24.2 tps (4.8× vanilla), finishes in 2.6× less wall time, and uses 62% less energy, at the coolest DDR (54.4 °C) of any policy. The cost is verbatim recall of the evicted spans; on a disjoint slice μ KV holds full-cache perplexity, and on retrieval it pays no quality cost (§4).

Composition with 1-bit weights (Bonsai-8B). The two compressions are orthogonal by construction (§??): 1-bit weights collapse the static S_{weights} term, μ KV collapses the growing S_{KV} term, so we test them stacked on PrismML's Bonsai-8B (Table 1, bottom block). On the same 16K-decode workload μ KV composes cleanly: 4.5× decode throughput (4.5 vs 1.0 tps), 60% less energy (2312 vs 5758 mWh), a retained live KV of 111 vs 1417 MiB, and a 9.7 °C cooler peak DDR—with peak skin 6.3 °C cooler and decode-phase CPU ~6 °C cooler, though the shared compute-bound prefill pins both to the same ~79 °C CPU peak—all with no change to μ KV's mechanism or hyperparameters. Figure 6 shows why this matters under sustained load: vanilla runs 85.6 minutes and drives the SoC onto a 50 °C battery / 53 °C skin plateau. The moment the battery crosses 50 °C (at 40 minutes) the vendor thermal engine deep-throttles *both* clusters together, the prime cores to 883 MHz and the perf cores to ~1000 MHz in a sawtooth (125 s sustained below 1 GHz); this 50 °C battery threshold, not the clock, is the throttle trigger. Both policies run the same vendor-clamped clocks while active, so μ KV avoids the throttle purely by finishing in 33.2 minutes at a 44.2 °C battery peak, before the phone reaches that threshold. One honest boundary: the FA-on-evict decode keeps the physical cache sparse (peak footprint ~1.8 GB, holes left by seq_rm unreclaimed), so the win here is decode *bandwidth*, from attention reading only the ~2K live cells, not resident memory; the state-swap variant that physically defragments (§4) is evaluated on Llama (top block), with its Bonsai counterpart left to future work.

Retrieval quality (NIAH). The needle-in-a-haystack sweep covers 8 policies × 3 models × 7 depths × 2 contexts (336 cells). μ KV-mass wins the multi-axis Pareto on every model and context: it attends to its 1024-cell target versus 3000–3700 for the score-reading evictors, runs 3–3.6× faster (it stays on the FA-on path while they stay FA-off), holds DDR peaks at or below 51 °C, and matches or beats vanilla hit rates. StreamingLLM fails retrieval on every model (its sliding window evicts exactly the middle of the context, where buried facts live).

Reading hit rates needs care: several misses are model-level recall limits, not eviction failures. On Llama-1B 8K, μ KV-mass and TOVA-canonical tie at 7/7 and both beat vanilla at depth 17%, because aggressive attention pruning acts as a distractor filter that sharpens the query–needle alignment; vanilla itself misses that depth. Where every policy including vanilla misses the same depth (Gemma-2B at 0% and 17% of 8K prompts), the model, not the evictor, is the cause. The controlled μ KV-count vs. μ KV-mass comparison isolates α_a : the mass split's larger recent budget gives the model the query-anchoring context it needs, which is why we default to it.

Measurement notes for Table 3. The prompt embeds the needle at depths $d \in \{0, 17, 33, 50, 67, 83, 100\}$ % of context; a hit means the 64-token generation contains the needle. Wall is the median end-to-end time per cell and is prefill-dominated on this short-decode

Table 3: Phone NIAH retrieval, all 392 cells (7 policies $\times$ 4 models $\times$ 7 depths $\times$ 2 contexts). One build, ctx 16384, cold-start gate on every cell. Hit: needle named as an answer, out of 7 depths. ActKV: mean retained cache — comparable *within* a block only, since per-token KV differs 24 $\times$ between Phi-3 and Llama-1B. Energy: USB rail per cell. The watchdog ran on every μ KV cell and never fired (battery peaked at 33.7 °C against a 47 °C first tier), so these are the cache lever alone.

Block	Policy	Hit	TPS	Wall (s)	ActKV (MiB)	DDR (°C)	Energy (mWh)
Phi-3 @ 4K	vanilla	7/7	5.8	127	1393	62	236
	μ KV	7/7	9.0	134	306	64	248
	SnapKV	7/7	4.1	154	1131	63	284
	AdaKV	7/7	4.2	153	1095	62	281
	TOVA	7/7	3.9	151	1196	61	276
	H2O	7/7	3.8	178	1212	61	292
	StrLLM	2/7	4.2	169	337	62	283
Phi-3 @ 8K	vanilla	7/7	2.2	418	2713	60	575
	μ KV	7/7	9.6	412	312	60	567
	SnapKV	7/7	2.1	477	1390	59	652
	AdaKV	7/7	2.1	479	1459	60	649
	TOVA	7/7	2.0	468	1653	60	640
	H2O	7/7	1.9	480	1297	60	651
	StrLLM	1/7	2.1	467	313	60	640
Llama-1B @ 4K	vanilla	7/7	24.1	42	98	55	57
	μ KV	7/7	35.6	54	25	54	61
	SnapKV	7/7	22.0	67	97	54	73
	AdaKV	7/7	19.5	76	74	53	75
	TOVA	7/7	9.7	81	83	49	78
	H2O	6/7	14.4	76	98	51	77
	StrLLM	2/7	11.6	92	26	49	79
Llama-1B @ 8K	vanilla	6/7	12.8	109	191	57	151
	μ KV	6/7	37.4	94	25	58	146
	SnapKV	5/7	15.2	137	171	59	183
	AdaKV	5/7	15.5	143	99	58	186
	TOVA	5/7	8.0	216	115	51	195
	H2O	2/7	11.8	128	164	55	336
	StrLLM	1/7	10.2	239	31	52	201
Gemma-2B @ 4K	vanilla	6/7	12.0	95	320	59	152
	μ KV	6/7	18.1	136	73	57	158
	SnapKV	6/7	11.3	86	317	57	112
	AdaKV	6/7	10.4	98	281	57	114
	TOVA	6/7	6.6	154	278	52	127
	H2O	6/7	7.0	154	320	53	130
	StrLLM	1/7	10.3	107	86	56	115
Gemma-2B @ 8K	vanilla	5/7	6.4	307	623	58	415
	μ KV	5/7	15.3	307	73	58	409
	SnapKV	5/7	7.7	348	620	60	314
	AdaKV	5/7	7.3	251	380	60	296
	TOVA	5/7	5.7	264	451	57	296
	H2O	4/7	6.1	210	623	58	285
	StrLLM	1/7	7.5	228	103	60	296
Bonsai-8B @ 4K	vanilla	7/7	4.5	146	448	62	259
	μ KV	7/7	8.4	169	109	61	263
	SnapKV	7/7	4.5	229	448	59	304
	AdaKV	7/7	4.4	324	415	56	334
	TOVA	7/7	3.3	347	434	54	590
	H2O	7/7	3.8	320	448	56	338
	StrLLM	2/7	4.4	312	124	57	331
Bonsai-8B @ 8K	vanilla	7/7	2.3	469	875	60	624
	μ KV	7/7	6.9	461	107	60	602
	SnapKV	7/7	2.5	603	868	59	690
	AdaKV	7/7	2.5	654	709	59	712
	TOVA	7/7	2.3	721	725	58	753
	H2O	6/7	2.2	605	868	59	688
	StrLLM	1/7	2.4	701	118	58	742

Table 4: LongBench F1 (official token-F1, max over reference answers) on the RTX 4500 Ada. Every policy runs f16 K/V here, so F1 and retained cache are quantization-matched: SnapKV cannot use quantized KV on this engine at all, since a per-head evictor needs FA-off and llama.cpp requires flash-attention for a quantized V. Cells is retained cache in KV cells (format-independent) as a percentage of the prompt. SnapKV runs its published LongBench configuration (window 32, avgpool-7); its NIAH setting (16/5) and the FasterDecoding default (64/5) were also measured and span only 0.8 F1 on Llama-1B, so the comparison does not turn on that choice. Gemma-2-2B runs at ctx 8192, its trained context; the other models at 16384. [†]StreamingLLM runs its OWN published budget, start_size 4 + recent_size 2000 = 2004 cells (its repository’s argparse defaults), not the $K=1024$ every other row uses. An earlier version of this table ran it at 1024 — roughly half the cache its authors specify — which cost it 1.7–4.6 F1 and reported it at $-6.1/-7.4/-10.8/-9.6$ instead of the $-4.4/-4.2/-6.2/-7.4$ here. Its retained cells rise correspondingly, so it is no longer at μ KV’s retention: μ KV holds $2.4\times$ less cache and still scores 4–7 F1 higher. Re-measured 2026-08-16, same 50 samples, same prompts, same binary; only $--k$ -nominal changed.

Model	Policy	HotpotQA	2WikiMQA	MultiFieldQA	Qasper	TriviaQA	Avg	Cells kept
<i>Llama-1B (ctx 16384)</i>								
	vanilla (full cache)	42.14	23.86	45.04	17.55	81.19	41.96	8687 (100.0%)
	μ KV	40.72	25.53	43.23	17.06	83.88	42.08 (+0.1)	826 (13.0%)
	SnapKV	41.14	23.51	45.02	22.81	81.19	42.73 (+0.8)	6717 (82.4%)
	Ada-KV	41.84	22.17	46.16	22.10	81.19	42.70 (+0.7)	3327 (47.3%)
	TOVA	41.61	22.98	45.62	21.93	81.86	42.80 (+0.8)	3824 (53.5%)
	H2O	42.68	24.04	41.87	19.35	81.20	41.83 (-0.1)	6699 (83.1%)
	StreamingLLM [†]	35.61	18.14	36.52	16.51	81.00	37.56 (-4.4)	1994 (23.0%)
<i>Gemma-2B (ctx 8192)</i>								
	vanilla (full cache)	46.51	34.51	45.19	36.42	87.67	50.06	6457 (100.0%)
	μ KV	40.32	37.20	39.63	31.71	87.67	47.31 (-2.8)	716 (13.0%)
	SnapKV	40.51	36.81	45.40	38.38	87.44	49.71 (-0.4)	6031 (94.2%)
	Ada-KV	40.51	36.81	44.55	37.26	87.44	49.31 (-0.7)	4064 (66.3%)
	TOVA	41.52	37.31	44.37	37.45	87.44	49.62 (-0.4)	4087 (66.9%)
	H2O	37.38	37.31	43.61	35.37	87.44	48.22 (-1.8)	5869 (92.1%)
	StreamingLLM [†]	41.51	32.67	38.72	28.59	87.67	45.83 (-4.2)	1995 (30.9%)
<i>Phi-3 (ctx 16384)</i>								
	vanilla (full cache)	46.86	43.51	57.05	36.92	87.87	54.44	9702 (100.0%)
	μ KV	45.97	43.18	55.85	40.80	86.37	54.43 (-0.0)	853 (12.2%)
	SnapKV	46.85	40.01	55.62	38.35	87.20	53.61 (-0.8)	7629 (83.4%)
	Ada-KV	46.90	38.48	56.51	37.33	87.87	53.42 (-1.0)	4056 (50.6%)
	TOVA	54.81	38.67	55.33	50.60	87.87	57.45 (+3.0)	4550 (54.9%)
	H2O	48.32	39.66	56.12	36.97	88.20	53.85 (-0.6)	8702 (91.2%)
	StreamingLLM [†]	40.71	41.88	40.05	30.68	87.87	48.24 (-6.2)	1999 (20.6%)
<i>Bonsai-8B (ctx 16384)</i>								
	vanilla (full cache)	35.11	33.39	50.03	38.53	86.15	48.64	8928 (100.0%)
	μ KV	35.16	24.33	48.13	32.30	84.95	44.97 (-3.7)	797 (12.6%)
	SnapKV	38.58	34.30	50.12	38.59	88.15	49.95 (+1.3)	8716 (96.1%)
	Ada-KV	34.46	34.48	50.75	37.56	87.48	48.95 (+0.3)	5293 (68.2%)
	TOVA	37.06	30.19	50.76	37.88	87.48	48.67 (+0.0)	5700 (72.5%)
	H2O	36.66	28.54	49.03	34.97	88.15	47.47 (-1.2)	7798 (90.9%)
	StreamingLLM [†]	34.40	24.97	30.13	30.24	86.26	41.20 (-7.4)	2246 (25.2%)

workload, so higher decode throughput does not lower wall here; the 2K-decode workload of Table 1 flips that ordering. ActKV is the end-of-prefill attended set. Temperatures are median peaks across depths because means skew on single-cell outliers (one H2O cell triggered a BCL event with a 93 °C prime-core spike). One Gemma-2B 8K μ KV-mass cell logged a 10.5 Wh USB-rail reading at normal wall and throughput, a clear sensor glitch, and is excluded from that energy mean.

Energy and the watchdog on NIAH. Integrating the USB rail over all 392 cells, μ KV is the lowest-energy policy at 307 mWh

and decodes at 17.6 tps against vanilla’s 8.8. Every other evictor costs *more* energy than the full cache — SnapKV 327, Ada-KV 331, StreamingLLM 336, H2O 350, TOVA 369 mWh — because they either read attention every decode step or cannot compact, so they pay the eviction cost without collecting the bandwidth saving. Against vanilla’s 309 mWh, μ KV’s 307 is a tie rather than a win: this workload generates only 64 tokens, so it is prefill-dominated and the $2.0\times$ decode advantage has almost nothing to amortise. The energy result is the 4K-decode workload of Table 1 (391 vs 1033 mWh), not this one. The watchdog ran on every μ KV cell and never fired: each cell starts from the cold gate and lasts about

Table 5: Mobile KV-cache work: what each paper *measured*, verified from its evaluation section rather than its title. The field splits cleanly: policies that *evict* run on servers or Jetson-class boards; systems that run on *phones* keep every token (offload, reuse, or quantize); and studies that do use a phone *GPU* apply no cache management at all. μKV occupies the empty cell.

System	Venue/yr	Hardware evaluated on	Phone	Compute used	KV mechanism
<i>KV eviction, but never on a phone</i>					
SnapKV, Ada-KV, H2O, TOVA, PyramidKV	'23–'24	A100-class servers	–	server GPU	eviction
KVSwap	MobiSys'25	Jetson Orin AGX 64 GB + NVMe	–	board GPU	offload to disk
MobiLoRA	ACL'25 [†]	NVIDIA AGX Orin	–	board GPU	cache reuse
KeyDiff	NeurIPS'25 [‡]	A100 80GB; unnamed Samsung phone (App. H.4)	• [§]	not stated	eviction
<i>On a real phone, but never eviction — and never the GPU</i>					
DynaKV	2025 [†]	OnePlus Ace5 Pro / 12 / Ace3 / Ace2	•	CPU only [¶]	offload to UFS
ActiveFlow	2025	OnePlus 12, Pixel 6, Infinix ZERO 30	•	CPU only	weights, not KV
PowerInfer-2	MobiSys'25	OnePlus 12, OnePlus Ace 2	•	CPU + NPU	neurons, not KV
PerCache	2026	Pixel 7, Redmi K60 Pro, S22 Ultra, Ace 6	•	not stated	QKV reuse (RAG)
Persistent Q4 KV	2026	MacBook Pro M4 Pro	–	Metal	quantize + persist
<i>On a phone GPU, but no cache management at all</i>					
LLMs in Your Pockets	2026	7 devices: Adreno 750/730, Mali-G720/G78	•	CPU, GPU, NPU	none
LLM Inference at the Edge	2026	S24 Ultra (Adreno 750), iPhone 16 Pro	•	GPU	none
μKV (this work)	–	OnePlus 15, Adreno 840	•	CPU and GPU	eviction + compaction

[†]co-authored with Honor Device Co. [‡]Qualcomm AI Research. [§]every model result (LongBench, NIAH, and the headline 30% end-to-end TTFT gain) is measured on A100; the phone appears only in App. H.4, which reports five *normalized* ratios for the eviction-scoring kernel — no model is run on the device, and no absolute latency, throughput, footprint or energy is given. [¶]“we adopt the CPU as the primary backend”; CPU–GPU is Discussion 2, future work.

four minutes, so battery peaked at 33.7 °C against a 47 °C first tier. Every number in Table 3 is therefore the cache lever alone, with the clock untouched.

4.1 Cache size is not a temperature actuator

We also tested coupling the cache budget itself to DDR temperature (a four-tier ladder over $K \in \{256, 512, 768, 1024\}$, symmetric and ratcheted variants, under cool and co-loaded regimes). The mechanism works, but the result is negative and we do not ship it: cache size controls throughput, wall time, and energy, not peak temperature. Halving K under co-load left peak DDR unchanged because the co-load dominates the thermal budget, and on a cool phone a smaller cache runs hotter at peak because the saved bandwidth converts to speed and power. Frequency remains the only lever that caps heat, which is why the shipped design pairs eviction (efficiency) with the clock watchdog (temperature) and nothing else.

4.2 Watchdog: gradual clock reduction before the kernel throttle

The watchdog’s target is *sustained, throttle-free operation*: keeping a long-running workload below the temperature at which the vendor deep-throttles, so the phone sustains the work at a stable equilibrium instead of hitting a cliff. It is a userspace daemon that samples battery, skin, DDR, and CPU at 2 Hz and lowers the big-core cap one step at a time, never raising it, running only under μKV. We anchor the ladder to the *measured* trigger: on this SoC the deep-throttle fires the instant battery reaches 50 °C (Fig. 6; the vendor holds a flat 1498 MHz right up to that point, then cliffs), so the watchdog stays dormant until the run-up window and then glides the cap below the vendor clamp—battery 47.0/48.0/48.5/49.0/49.5 °C and skin 50.0/51.0/51.5/52.0/52.5 °C map to 1497/1382/

1267/1132/1017 MHz—bending the temperature curve before it can reach 50 °C, with a 1017 MHz floor that stays above the 883 MHz cliff so it never self-throttles.

The orange trace of Figure 6 demonstrates this on the workload that actually reaches the cliff. Without the watchdog (grey) vanilla Bonsai-8B climbs to a 50.2 °C battery and the vendor deep-throttles both clusters to 883 MHz for 125 s. With it (orange) the glide—driven by *both* ladders, as battery crosses 47–49 °C and skin crosses its 50–52 °C rungs (skin panel)—flattens the battery rise from 0.48 to 0.05 °C/min and settles it at a stable 49.1 °C (skin 52.0 °C), below the 50 °C trigger, so the deep-throttle collapses from 125 s to 1 s. The DDR panel of that figure isolates what the clock lever can and cannot do: DDR peaks near 70 °C in *both* vanilla runs (the sustained level is set by memory traffic, i.e. cache size), yet the clock still modulates it—the no-watchdog DDR sawtooths in lockstep with the throttle, each 883 MHz dip shedding ~8 °C of DDR, while skin, with far more thermal mass, tracks it only weakly. So both cache and clock are thermal levers: the watchdog removes the throttle with the clock, while μKV lowers the whole DDR level by cutting memory traffic.

Two clarifications. First, the mild vendor step from 1632 to 1498 MHz near 40 °C skin is ordinary DVFS, not throttling; we reserve the term for the deep drop toward 883 MHz. Second, because this decode is bandwidth-bound the clock is not the throughput limiter, so preventing the throttle adds no speed—vanilla runs at 1.0 tps and 86 minutes with or without the watchdog. The watchdog therefore buys *thermal sustainability*, not throughput: it is complementary to μKV, which owns speed and efficiency through cache compaction. In the cold-start systems runs of Table 1, μKV finishes fast enough that no policy reaches a deep throttle at all, so its advantage there is efficiency (2.6× less time, 7 °C lower skin); the watchdog is the insurance for the sustained-hot regime, where

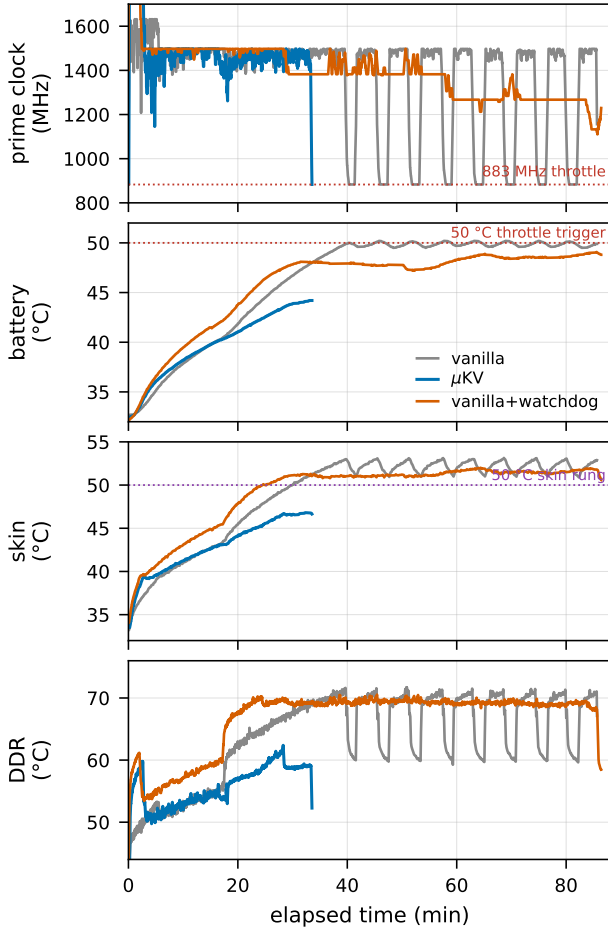


Figure 6: Two levers against the sustained throttle (Bonsai-8B, cold start): prime clock, battery, skin, and DDR for vanilla, μ KV, and vanilla+watchdog. The watchdog is part of μ KV and is never given to a baseline in any comparison; the vanilla+watchdog trace is an *ablation* only, included because μ KV finishes before reaching the cliff and so cannot show the clock lever in isolation. Dotted lines mark the 883 MHz throttle floor and the 50°C battery/skin thresholds; the cache and clock levers are analyzed in §4.2.

a workload would otherwise cross 50°C, and the orange trace of Fig. 6 shows it holding exactly that line.

4.3 Energy-aware scheduling: three levers, one lever, two loops

The watchdog holds temperature. A second controller, added last, holds energy: it picks, per request and from the battery state the phone reports, the GPU clock cap, the cache budget and the output cap. We first measured what each control buys on this phone, then built the scheduler on those measurements, then provoked its loops with real disturbances, then ran it through a real discharge. Figure 1 shows the whole mechanism, steps 5 to 8. Every number

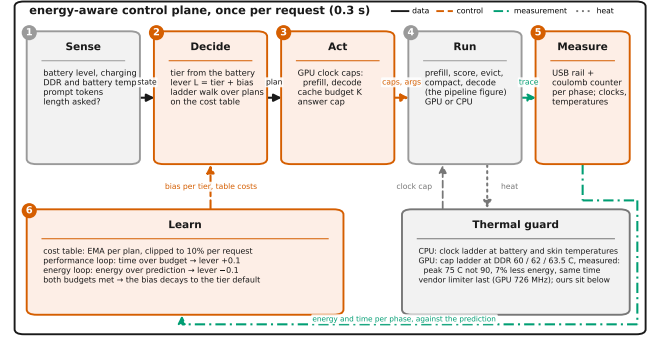


Figure 7: The energy-aware control plane: sense, decide, act, run, measure, learn, once per request, with the thermal guard beside it. Solid: data. Dashed: control. Dash-dot: measurement. Dotted: heat.

below is from the OnePlus 15 with Llama-3.2-1B and a 9737-token prompt.

What each control buys. The GPU clock cap trades prefill energy for prefill time: 1200 to 902 MHz cuts a 4096-token request from 1428 to 1154 J for 221 to 263 s, and the next step to 726 buys 3 percent more energy for 21 percent more time, so it is never taken. Decode is bandwidth-bound and barely feels the cap, so a split plan, prefill at 1200 with decode capped at 902, keeps the prefill speed and saves 6 percent of the request for 1.5 percent of time; it is the best measured point by energy-delay product, and it is the two-configuration schedule that Kim, Imes and Hoffmann prove optimal under a deadline [29]. The CPU clock is not an energy lever: energy per token is flat within 7 percent from 883 to 1632 MHz, because power is close to linear in frequency at the pinned voltage, so the scheduler never touches it and leaves it to the watchdog. The GPU also gets a ladder now: a daemon steps the user cap to 1050, 967 and 902 MHz at DDR 60, 62 and 63.5°C, below the vendor’s trip. On the 4096-token request at 1200 MHz, where the vendor fires in every cell, three cooled cells per arm give 1289 against 1390 J, 216 against 214 s and a DDR peak of 75 against 90°C: 7 percent of energy and 15 degrees for 1 percent of time, which reverses our earlier GPU-watchdog finding for any request long enough to reach the limiter. The cache is the CPU lever only against the full cache, 62 percent less energy and 4.6 times faster at $K=1024$; below 1024 the tiers save 3 to 4 percent of a request while the worst task loses 37 to 42 percent of its score, so K stays at 1024. Output length is linear in tokens at 0.17 to 0.21 J per token above a fixed prefill cost, which makes the answer cap the largest lever when the caller left the length open.

One lever, two loops. Figure 7 lays the loop out. A lever $L \in [0, 1]$, set by the battery tier (1 above 50 percent or on mains, 0.5 at 21 to 50, 0 at 20 or below), fixes every rate the scheduler trades at: the exchange rate a shared step must beat (1.5, 0.8, 0.5, energy saved per unit of time given up), the quality floor, the time slack and the output cap (4096, 1024, 512). A measured cost table predicts energy and time per plan; the walk down the ladder takes a step only if it beats the exchange rate and stays inside the time budget. Two

Table 6: What the scheduler does at each battery tier and what it costs: cooled cells on the cable, and the real discharge on the battery. Llama-3.2-1B, 9737-token prompt, OnePlus 15.

battery	GPU plan	cap	cable, J	batt
mains or above 50%	1200, decode 902	4096	1336 (1428 uncapped)	
21 to 50%	1200, decode 726	1024	577 (at 902)	
20% or below	902	512	500	
GPU unavailable	CPU, $K=1024$	by tier	822 (1024 tokens)	

loops then act on model error, the JouleGuard coupling [28]: the performance loop nudges the lever toward performance when a request runs over its time budget, the energy loop toward energy when it draws more than its prediction, both with a 5 percent tolerance, and the bias decays when both budgets are met. Knobs only one loop feels are assigned to it, the decode clock and the cap to the energy loop, the CPU clock to the watchdog; only the prefill clock is shared, and the lever arbitrates it. A decision costs 0.3 s on the phone. A contextual bandit trained on 24 real requests picks the same plan in every tier (13 of 15 free choices were the best arm, none strictly wrong), so the rule is not arbitrary; it is also what the bandit cannot do that matters, since it holds one reward per arm and cannot transfer to a new prompt shape or notice a regime change without a fresh warm start.

The loops, provoked. Thirteen cooled requests with real disturbances (Figure 8). Four idle cores spinning through a mid-tier request pushed its energy 30 percent over budget; the energy loop took the lever down twice, the walk reached the 902 cap, and two clean requests decayed the bias back. An external 726 MHz cap written 12 s into three low-tier requests, what the vendor limiter does, put them 10 to 12 percent over their time budget; the performance loop took the lever up three times to its clamp, then the energy loop pulled it back when the cap was gone. The phone contributed a disturbance of its own: its limiter trips at DDR 64 °C and drops the GPU to 726 mid-decode, and the loops handled it. The visible weakness was that the table learned interference as if it were a plan’s cost; learning is now clipped to 10 percent per request.

The real discharge. A loop resident on the phone ran 123 requests through the scheduler from 91 to 11 percent with charging disabled and no forced state (Figure 9). The scheduler switched on the first request past each threshold: the healthy plan with 4096 tokens down to 52 percent, the mid plan with the 1024 cap from 50, the low plan from 19. On the battery the tiers cost 885, 748 and 506 J per request in 280, 210 and 177 s: 15 and 43 percent less energy than the healthy tier, with time falling 25 and 37 percent. Two findings came with it. On the battery the phone is a different machine from the phone on the cable: the same healthy request costs 885 J instead of 1274 and takes 280 s instead of 218, because the OEM runs the GPU cooler when unplugged and its limiter caps prefill at 726 MHz within a minute of every request. There the answer cap and the prefill cap carry the saving and the decode-clock lever is pre-empted by the vendor; per token, healthy and mid cost the same 64 mJ and low 49. And a table seeded from cable cells over-predicted battery requests by 36 percent until the clipped learning

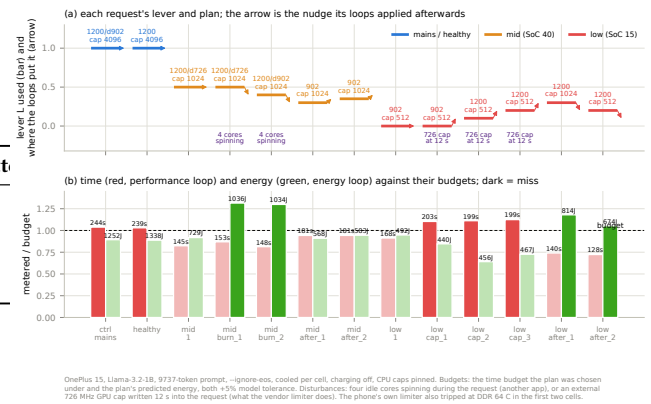


Figure 8: The two loops on the phone: each request’s lever and plan with the nudge applied after it (top), metered time and energy against their budgets (bottom). Dark bars are misses.

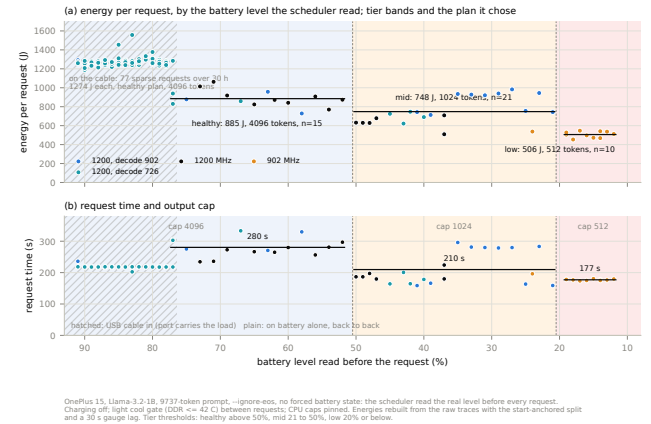


Figure 9: The scheduler on a real discharge: energy and time per request against the battery level it read, with tier bands, the plan chosen and the cap. Hatched: USB cable in.

brought it back within 2 percent, in three requests; over the run its mean error was 8.9 percent on the battery against 3.7 on the cable, the limiter’s variable trip time being the difference. No shipped phone changes an LLM’s clock, context or answer length by battery state, and the closest research system excludes battery level by design [30]; the only per-rail energy dataset on this phone, PowerBench [31], and the FUSE study of mobile governors [32] agree with our per-token costs and with the finding that a profiled static setting matches learned governors on a known workload.

4.4 Cross-model replication of the throughput-energy Pareto

The headline Phi-3 result replicates on Llama-3.2-1B and gemma-2-2b-it (Table 1, bottom 6 rows). On Llama-1B at 16K decode, μ KV reaches **16.29 tps** (vs vanilla 4.87, AdaKV 5.90) at **446 mWh** system energy (vs vanilla 634, AdaKV 653). On gemma-2-2b, **7.17 tps**

(vs vanilla 3.35, AdaKV 2.69) at **847 mWh** (vs 1322, 1367). The pattern is consistent: μ KV is the fastest and the lowest-energy policy on every model, holds DDR 8–10 °C below the cliff vanilla and AdaKV approach, and uses the least RSS. PPL on Llama-1B and Gemma-2B shows the count- α saturation effect (Sec. 3, the dispatcher v1) over-allocating anchors for smaller models; the mass- α dispatcher rebalances toward recent on these regimes, closing the quality gap in follow-on sweeps.

4.5 Vision-language models

Image tokens enter the same KV cache as text tokens, so μ KV applies to a vision-language model without any change to the policy. We ran Qwen2.5-VL-3B with one image and a 1024-token answer. μ KV cut the live cache from 144 to 23 MiB, a 6.3 $\times$ reduction, and kept prefill and decode within 1% of the full cache. The answers stayed correct.

One engine change was needed. Multimodal position encoding gives every token of an image the same position index: in our run 4105 cache cells shared 110 positions. The cache API in llama.cpp removes cells by position range, so it cannot express a per-token keep mask on such a cache, and applying one silently corrupts the cache instead. We added a cell-index removal call and use it only when the cache is multimodal. The text path is untouched and still produces bit-identical selections.

At this size the cache is small next to the model weights, so there is no speed gain here; the win is memory. Multi-image and video inputs, where the cache grows into gigabytes, are the case where we expect throughput to follow, and that is the workload the disk-swapping systems (§??) target.

4.6 Limitations and scope

Device. All measurements on a single OnePlus 15 (Snapdragon 8 Elite Gen 5). Watchdog thresholds are device-specific; the calibration protocol (10 measured pre-throttle events on the vanilla baseline) generalizes but the specific tier values do not. *Energy.* System energy via USB rail $V \cdot I$ integration with charging disabled throughout each cell. This is total-device energy (CPU, DDR, modem, screen-off); we cannot isolate DDR-specific energy without a die-attached power meter.

Why CPU-only. Sustained decode on the mobile GPU is bandwidth-saturated, so eviction has little throughput headroom to recover there; the CPU is where a bounded cache converts directly into speed and energy. The GPU results (§4) show the thermal and energy story still holds, and the realizability finding is engine-level, not processor-level.

Portability. The method needs privileged thermal and power telemetry, available on rooted Android. On iOS the same controller would run on coarser public sensors; the eviction mechanism is OS-independent and only the watchdog's calibration is device-specific.

5 DISCUSSION: PICK THE POLICY PER REQUEST

The tables above no longer split along the axis we originally expected. With the count-based dispatcher, retrieval was the workload class where Ada-KV justified its FA-off cost; with the mass-based α_a dispatcher, μ KV ties the full cache block-for-block across all 392 NIAH cells (Table 3) while remaining 1.4–3 $\times$ faster and cooler, and preliminary LongBench sample-0 F1 shows the same ordering (full $n=5$ sweep in progress). The split that survives the data is instead *prediction* vs. *verbatim recall*. On a disjoint slice the evicted cache predicts as well as the full one (within 2% perplexity on all three models), so the full cache's remaining edge is narrow: reproducing text it still holds, at 5.5 $\times$ the wall and twice the energy. The design question is therefore not which evictor wins, but how the phone should set K per request: large for long free-form continuation when cool, small everywhere else — exactly the admission-control role that the budget ladder + the admission check already occupy (§3).

A request whose output entropy stays high is probably searching for a fact and should not be evicted aggressively; one that commits quickly can be. We are building a per-step entropy gate $K_h(t) = \text{round}(K_{\text{nom}} \mu(m_h) (1 - \tilde{H}(t)))$ that shrinks the budget when normalized output entropy $\tilde{H}(t)$ is high. A server-side study over long-form tasks shows the expected negative correlation between output entropy and attention concentration ($\rho = -0.37$, $p < 10^{-55}$), the relationship the gate exploits. A second extension spills evicted cells to flash with an index for linear-time recall, extending the watchdog to flash temperature and I/O depth, so model-internal attention drives both eviction and recall while sensors drive both the frequency cap and the I/O quota.

Limitations. All numbers are on one device; the watchdog thresholds are device-specific. Energy is a disable-charging estimate, not a power-meter measurement, so absolute mAh should be read within about 10% while the sign and ordering hold. μ KV is faster than the full cache on long prompts and no longer slower on short ones: in-graph scoring removes the prefill penalty, and only the 290 ms compaction remains to amortize. All measurements use seed 42; cross-seed variance is in the supplementary material.

6 CONCLUSION

μ KV is a mobile inference pipeline that makes attention-driven KV eviction viable on a phone and measures it end-to-end on a OnePlus 15: a cheap $O(H)$ per-head budget signal, in-graph scoring with sequence-level compaction that keeps the evictor on the fast path, a cliff-insurance frequency watchdog, and thermal-coupled cache control. 08 tps where μ KV holds 5.18. We honestly report what μ KV does not do — no configuration of the cache actuator caps peak temperature under external co-load (frequency remains the only lever that does); the symmetric ladder oscillates and is retired in favor of the measured ratchet (+47% tps, −23% energy, −40% time-above-cliff at identical PPL under identical co-load, the 10-arm test); and eviction still costs verbatim recall of the spans it drops, even though it preserves both prediction (full-cache perplexity on a disjoint slice) and retrieval (7/7 NIAH at 4K on every model). The useful takeaway is the substrate, not the trophy:

on-device LLM serving should close the loop between attention-driven cache state and hardware sensors per request — the cache is the phone’s *efficiency* actuator, the clock its *temperature* actuator, and the two compose.

Adaptive dispatcher. The mass-based α_a removes the deploy-time corner choice. On Llama-1B at $K=1024$ it matches the manually tuned quality corner on qasper (7.38 F1 either way), auto-discovering the same anchor-heavy allocation the sweep found; at the tighter $K=512$ it lifts qasper 3.69 to 5.87 F1 (+59%) and hotpotqa 5.11 to 10.24 F1 (+100%), and recovers phone NIAH from the broken 0/14 baseline to every measured cell, with no per-model recalibration.

Reproducibility. We release the eviction_bench binary with all six policy implementations, the multi-sensor watchdog daemon, the energy fallback chain, and the host-side harnesses for Wave-11 chunk-pair PPL, NIAH Tier-1, and LongBench at [anonymizedURL]. Phone-side scripts target Android NDK aarch64; host-side LongBench builds against llama.cpp with CUDA. All measurements use seed 42 throughout; the protocol scripts pin DVFS to the performance governor and the kernel cap (1632 MHz on Snapdragon 8 Elite Gen 5) before each cell. All raw meta.json and stress.csv artifacts referenced in the tables are included in the release.

REFERENCES

- [1] Z. Zhang et al. “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs.” NeurIPS, 2023.
- [2] G. Xiao et al. “Efficient Streaming Language Models with Attention Sinks.” ICLR, 2024.
- [3] M. Oren et al. “Transformers are Multi-State RNNs.” arXiv:2401.06104, 2024.
- [4] Y. Li et al. “SnapKV: LLM Knows What You’re Looking For Before Generation.” NeurIPS, 2024.
- [5] Z. Liu et al. “KIVI: Tuning-Free Asymmetric 2bit Quantization for KV Cache.” arXiv:2402.02750, 2024.
- [6] Y. Feng et al. “Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation.” NeurIPS, 2025.
- [7] Y. Feng et al. “CriticalKV: A Perturbation Perspective on KV Cache Eviction.” ICML, 2026.
- [8] Y. Feng et al. “DefensiveKV: Worst-Case Risk Control for KV Cache Eviction.” ICLR, 2026.
- [9] Y. Gu et al. “AhaKV: Adaptive Holistic Attention-Driven KV Cache Eviction.” arXiv:2506.03762, 2025.
- [10] C.-P. Hsieh et al. “RULER: What’s the Real Context Size of Your Long-Context Models?” arXiv:2404.06654, 2024.
- [11] Y. Bai et al. “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding.” arXiv:2308.14508, 2023.
- [12] T. Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention.” NeurIPS, 2022.
- [13] T. Dao. “FlashAttention-2: Faster Attention with Better Parallelism.” arXiv:2307.08691, 2023.
- [14] Y. Song et al. “PowerInfer-2: Fast LLM Inference on a Smartphone.” arXiv:2406.06282, 2024.
- [15] K. Alizadeh et al. “LLM in a Flash: Efficient Inference with Limited Memory.” arXiv:2312.11514, 2023.
- [16] G. Gerganov. “llama.cpp.” <https://github.com/ggerganov/llama.cpp>.
- [17] Qualcomm. “Battery Current Limit (BCL) for Snapdragon Platforms.” Whitepaper, 2023.
- [18] P. Michel et al. “Are Sixteen Heads Really Better than One?” NeurIPS, 2019.
- [19] E. Voita et al. “Analyzing Multi-Head Self-Attention.” ACL, 2019.
- [20] J.-H. Kim et al. “KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction.” NeurIPS, 2025.
- [21] K. Liu, L. Zeng, L. Xu, B. Yang, Z. Yan. “PerCache: Predictive Hierarchical Cache for RAG Applications on Mobile Devices.” arXiv:2601.11553, 2025.
- [22] MLC Team. “MLC-LLM: Universal LLM Deployment.” <https://github.com/mlc-ai/mlc-llm>.
- [23] S. Kim et al. “zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices.” MobiSys, 2021.
- [24] S. Liu et al. “FUSE: Fine-Grained Power Management for Mobile Devices.” SenSys, 2022.
- [25] X. Cai et al. “R-KV: Redundancy-aware KV Cache Compression for Reasoning Models.” arXiv:2505.24133, 2025.
- [26] X. Zhou et al. “DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs.” arXiv:2412.14838, 2024.
- [27] PrismML. “Bonsai: Commercially Viable 1-bit LLMs.” 2026. <https://prismml.com/news/bonsai-8b>.
- [28] H. Hoffmann. “JouleGuard: Energy Guarantees for Approximate Applications.” SOSP, 2015.
- [29] D. H. K. Kim, C. Imes, and H. Hoffmann. “Racing and Pacing to Idle: Theoretical and Empirical Analysis of Energy Optimization Heuristics.” CPSNA, 2015.
- [30] B. Zou et al. “EnerInfer: Energy-Aware On-Device LLM Inference.” arXiv:2606.23001, 2026.
- [31] G. Cai et al. “Is Your NPU Ready for LLMs? A Power Benchmark of On-Device Inference.” arXiv:2607.05475, 2026.
- [32] Z. Zhang et al. “Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency.” arXiv:2507.02135, 2025.